FRED TALKS

4th June 2026

CONTENTS

Semantic Search Without Embeddings

DOUG TURNBULL · 08 JAN 2026 · SOURCE

When I need to stay warm, I search for “long johns” or “long underwear”. But modern outdoors clothing stores label this a “base layer”. For a waterproof jacket I search for “slickers” or a “ski jacket”, not realizing what I should search for is a “shell.”

Despite my outdated terminology, search still works. I’m somehow understood and shown the right content.

We call this **semantic search**. When you hear that, you might think embeddings. Today I want to stretch your thinking beyond. Fantasize and semanticize.

Some teams wrongly assume semantic search equates to only “vector search”. You have more options, some that might better align to your domain and existing organizational skills. LLMs simplify classic ways of organizing search queries and information.

Let’s contrast different approaches, and maybe you’ll see there’s another approach that might be a better fit for you.

Semantic search - what exactly is it?

In semantic search, content and query map to a ***shared representation***. This space has a ***similarity*** function that scores similar items higher than less similar.

Let’s walk through an example.

A user searching “round red fruit that grows on trees” probably wants an apple.

Humans know this - search engines don’t. So we need to help.

We map the query to three properties: `[round, red, fruit]`

In our search index we’ve also mapped three items into that space:

An **apple:** `[round, red, fruit]`

An **orange:** `[round, orange, fruit]`

A **baseball:** `[round, white, ball]`

Now we have a shared *representation*: a three tuple of (`shape` , `color`, `item type`).

How do we measure similarity? For each property (`shape`, `color`, `item type`) we could score a “1” if they’re equal, otherwise a 0. Then add to get a total score:

Similarity(query, apple) = 1 + 1 + 1 = 3

Similarity(query, orange) = 1 + 0 + 1 = 2

Similarity(query, baseball) = 1 + 0 + 0 = 1

So the user sees search results

1. Apple (Score 3)
2. Orange (Score 2)
3. Baseball (Score 1)

Ok, so a fairly silly method of semantic search. It ignores properties that matter (size, for example). It requires us to comprehensively tag everything. Still many teams try to tag + synonym their way to better search, with middling results.

Embeddings for semantic search

The default answer to the “semantic search” question is embeddings. The Internet is littered with documentation for how vector search works. I’ll do the speedrun here (references here).

We can learn a representation, an embedding, for apple and orange so that `similarity(apple, orange) > similarity(apple, baseball)`.

We learn an embedding from training data. We might declare when items get clicked for the same query, they are similar. We notice both “apple” and “orange” get clicked for query “fruit that grows on trees” at high frequency:

```
[fruit that grows on trees 🔍 ]
1. 🍎 (clicked)
2. 🥕
3. 🍊 (clicked)
```

We start with a random vector for each. A list of random floating points numbers like `[0.01, -0.1, 0.05]`. Then we train, in this super-secret-ultra-high-tech algorithm

```
for session in sessions:
    together = []
    for lhs in session: # ie the apple
        for rhs in session: # ie the orange
            if lhs != rhs:
                                nudge_closer(lhs, rhs) # ie nudge a
```

(Not shown: nudging away the negative, dissimilar items)

After training, for many queries+sessions, maybe apple becomes `[0.1, -0.2, 0.05]` while orange becomes a similar vector: `[0.09, -0.21, 0.04]`. We measure the similarity with cosine similarity, euclidean distance, and others. (We can replace "shared search session clicks" with any other notions of "similar": occurs in the same text passages, both occur in shopping carts, etc).

Learning an embedding per item presents problems. For example, maybe we just added strawberry 🍓 to our product listings. With no training data, the vector `strawberry_vector = embeddings['strawberry']` initializes to random garbage. It won't have a meaningful value until we've shown it to many users.

We'd prefer to to compute embeddings from ***features*** of the item. Then strawberry 🍓 could get an embedding right away. IE instead of `strawberry_embedding = embeddings['strawberry']` we can do `strawberry_embedding = f(heart_shaped, red, fruit)`. We build an ***encoder.*** Given the item's attributes we compute embeddings that produce the expected similarity. Embedding models downloaded from hugging face specialize in encoding specific type/domain of content for a specific task (search, etc).

We create encoders for all kinds of features: ordinal values (like a list of shapes, colors), the raw title / description text itself, images and so on. We can also train a two-tower model: one encoder for queries - ie "round fruit that goes on trees" - and another for documents.

Like other machine learning models we tradeoff precision for generality. We need the right features, we need enough training data for all the variations in those features (curse of dimensionality) and so on.

important thing is to embrace that and *as soon as possible, as often as possible, ship things on which we can get feedback on, in a way that gives us valuable feedback — by CI, by production, by our teammates, by select users, by select customers, by all of our users*.

Yes, italics ***and*** bold. Because building software is learning and we want to learn as much as possible.

- ... write the example code that would go into the README, show that around, does that look like a good API? People don't need an SDK built if they dislike the API in the readme.

- ...

There's more options that I didn't list here. And you don't have to pick only one. In fact, for something big, you should probably do a few of these things. Or you vary them, or combine them, or ...

What *exactly* you do doesn't matter as much as constantly asking: how can I get feedback on what I'm trying to build as soon as possible? And "feedback" here is used in the widest sense possible. Feedback comes in all shapes and sizes: feedback from the CI system on main, feedback from colleagues, feedback from users, feedback from *you* once you actually use it.

And if you follow that question — how can I get feedback as soon as possible, so I can learn? — you'll also find out how to chop things up and how to ship them:

- you won't get good feedback if you ship an "MVP" of an idea that's so obviously buggy that all you'll get is bug reports for 3 days, not actual feedback on how useful it is

- you won't get good feedback if the people supposed to give you feedback have to jump through 8 hurdles to test it

- you won't get good feedback if you keep your changes on a branch for 3 weeks, because by the time you merge your 27 commits and CI blows up you have 27 possible causes, vs. 1 if you had merged them one by one

- want feedback on the design of your skateboard? sure, show them the deck, no need for wheels. want feedback on how your skateboard feels? you can't ship it to testers without wheels on it.

- ...

So. Here's what I want you to think about going into this week: how can I get feedback on what I'm building on as fast as possible? when is the last time I got valuable feedback on what I'm building? in what frequency do I get feedback on what I'm building? why is that frequency so low?

Because we're building something new, in a time when software is changing (background vocals: *everything is changing*), and no one has a clue what the fuck is going on - so **the most**

The missing semantic search piece of vector retrieval

One pain point comes up right away with embeddings: lack of *matching*.

How would users actually react to these search results?

```
[fruit that grows on trees 🔍 ]
1. 🍎 Apple
2. 🍊 Orange
3. ⚾ Baseball.
```

In my experience, negatively.

It may have the perfect ranking. But search should exclude items outside what the user intended. They find fruit acceptable. But not baseballs.

So earlier I lied. What users actually want from semantic search:

- **Representation** - some space both queries and content can share representation

- **A similarity function** - A way of measuring how near / far items in the shared representation are from each other

- **Match criteria (new!)** - A system where an item can be said to be "match" or not for the query in the semantic space. It is the thing, or its not the thing.

In other words, I gave an incomplete **semantic search definition**. Embeddings thrive on the first 2, but don't do a great job at including / excluding results.

Understanding users (and their queries) matters a great deal in search. That requires including / excluding appropriate items based on terminology and categories familiar to users.

You might think "I'll just find a cutoff in my embedding similarity". Sadly, there's not a magical "at 0.8 similarity it's not a match". In some domains / queries one threshold works. In another, a very different threshold. Further, thresholding becomes tougher when users give multiple criteria. An embedding model might rank blood oranges above green apples for query "red apple". What matters more, the color or item type in the similarity?

Thresholds help exclude some obviously irrelevant results and improve performance, but they don't use domain-specific criteria, in the user's language, to include / exclude results.

The other semantic search approach (hierarchical managed taxonomies)

One system tries to solve all three problems: *Managed vocabularies* or *taxonomies*.

Taxonomies map queries + content to a hierarchy of concepts, using the language of the domain. Think a neatly organized directory tree. Or the Dewey Decimal System

Here's an example from the Wayfair WANDS furniture dataset of their category tree; Where you'd categorize "novelty rocking horses" apparently:

`Baby & Kids / Toddler & Kids Playroom / Indoor Play / Rocking Horses`

We can take a query for `hobby horse`. A human, curating a bunch of query rules, could manually map this to a node:

`Baby & Kids / Toddler & Kids Playroom / Indoor Play / Rocking Horses`

Assuming the corpus categorizes products correctly, we can search, filter, boost, etc using this.

We have a *representation* (the category tree), do we have a similarity function?

Yes, we can rank as follows:

1. Direct matches `Baby & Kids / Toddler & Kids Playroom / Indoor Play / Rocking Horses`
2. Parents / siblings. `Baby & Kids / Toddler & Kids Playroom / Indoor Play / Dollhouses`
3. Those rank higher than grandparents / cousins (``Baby & Kids / Toddler & Kids Playroom / Beanbag Chairs / …`

We can define meaningful match criteria too. We include shared parent categories (ie siblings). But exclude anything beyond cousins / shared grandparents as needed.

Many teams already maintain systems for organizing and cataloging information. For many domains, a good taxonomy can be a moat. Especially for technical domains (legal, medical, fashion, finance) where precise categorization really matters. Existing taxonomies already exists, and statistical fuzziness of embeddings can create huge downside.

fully specifying what you want. You can't avoid it, because you can't define it yet, because *building software is learning*!

Now that was the bad news. Here's the good ones. You *can* to reduce the time the `...` from the examples above takes — the time between the confident "yes, let me build this" and the humbled "oh, I see".

And that, in turns out, is **the most important thing you can do** when you're building something new: reducing the time it takes you to go from "let me try something" to getting your ass whooped by reality.

If someone says "we need this feature", don't go "yes, let me build it" and hack on something for 4 weeks only for the other person to ultimately go "that's not what I meant." No. Instead, embrace that *we need to learn*, that we need to try and play around with this idea as fast as possible, in a way that lets us learn. To embrace that idea means that you try to figure out "what is it they mean" *as fast as possible*, with the minimal effort required, so you can LEARN what it is you're building.

Instead of going away for 4 weeks and hacking on something, you instead can do stuff like this:

- ... build a prototype, in 1hr, and show it to them, and they go "no, that's not what I meant, you should change this part here"
- ... write down a spec of how you'd approach it, in 30min, and show it to them, and they go "no that's not what I meant"
- ... cut up the thing in multiple things and ship one every day, so that every day what you built hits reality and you get to learn, because on day 2 someone says "know what, we should change..."
- ... reduce the scope, figure out what the things are that we're already sure about and skip those, and instead focus on the bits that we don't know yet — that's where we need to learn. don't add 5 ways to login, if all we need to learn right now is 1.
- ... fake a demo video, show that around, get input on that - Quinn's done it many times
- ... write the news post that explains the idea — why waste effort building something for a week if the idea can be captured in 3 paragraphs?

That basically *NEVER* what happens. At least not when you're building something *new*. It might happen when you fix a bug or when you port something that already exists in another app to a new language or framework, ... Or when you're building after a spec.

But when there's no spec, and when you're building something new?

Here's how that works:

```
person a: “we need this feature”
person b: “yes, let me build it”
[...]
person b: “done.”
person a: “hmm, actually, that’s not what I meant. what I meant is: [
```

Or this:

```
person a: “we need this feature”
person b: “yes, let me build it”
[...]
person b: “you know what... There’s 3 ways to do this, actually, and
person a: “ah, I see, I think given that we want to ship this tomorro
```

Or:

```
person a: “we need this feature”
person b: “yes, let me build it”
[...]
person b: “done.”
person a: “Don’t like it”
```

Now why does that happen, again and again and again?

Because *building new software is learning*! If you're building something new and you don't yet fully know how exactly it's supposed to work, you will learn what exactly it is that you're building as you're doing it. Let me repeat: building new software is learning.

So far, so good, right? But here's the very important bit, the one bit I want you to take with you into this week: there is *no way in hell*, absolutely zero chance, that you can build something new *and* avoid bumping into "that's not what I meant", or "now that I'm working on it I'm not actually sure", or "hmm, now that I use it, I don't like it". Because the only way you could avoid that would be to fully specify what you want up front and, well, guess what programming is? It's

In the past, the management of a taxonomy would be massive. It might require complex rules, mapping of query phrases to taxonomy nodes, and a team of labelers and experts to keep everything straight. This has long been an appeal of embeddings.

But LLMs sneakily supercharge these old school ways of organizing information. Given a taxonomy, LLMs can easily + cheaply classify products and queries. It's more approachable to "fine tune". AI augmented labeling can simplify management of knowledge making it more approachable.

Semantic similarity doesn't need a vector index

Above I described a similarity function above (direct matches, then parents, then grandparents). With some creative tokenization, this similarity can happen in a normal BM25 index.

Consider our query for `hobby horse`. We know this maps to this node:

```
Baby & Kids / Toddler & Kids Playroom / Indoor Play / Rocking Horses
```

How do we score content in such a way so that direct matches rank higher than siblings, and so on?

We need is a hierarchical tokenizer. A function that produces the full path of every parent directory:

```
In: hierarchical_tokenizer("Baby & Kids / Toddler & Kids Playroom / I
Out: ['Baby & Kids',
      'Baby & Kids / Toddler & Kids Playroom',
      'Baby & Kids / Toddler & Kids Playroom / Indoor Play',
      'Baby & Kids / Toddler & Kids Playroom / Indoor Play / Rocking
```

If you index using this tokenizer, you'll recreate scoring needed for the direct > parent > grandparent similarity function I described earlier.

The key reason: BM25 rewards a rare match over common match.

Root nodes (ancestors) have higher document frequency, they're common. Common matches score lower in BM25.

That's because every item labeled `Baby & Kids / Toddler & Kids Playroom / Indoor Play / Rocking Horses` tokenized hierarchically has a `Baby & Kids` term. So does a document labeled `Baby & Kids / Kids Furniture ... / Beanbag Chairs`. `Baby & Kids` happens all over the place - maybe like 10% of the index.

Child nodes have low document frequency, they're rare. In BM25, rare, more specific matches outrank common ones.

A product labeled `Baby & Kids / Toddler & Kids Playroom / Indoor Play / Rocking Horses / Traditonal Rocking Horses` when tokenized will include the full path in the token. There might be 10 products in the entire index in this exact category. So that rareness - that high specificity - shoots this narrower, closer match to the top.

To do a deeper walkthrough

When I come along and search for

```
In: hierarchical_tokenizer(
     "Baby & Kids / Toddler & Kids Playroom / Indoor Play / Rocking H
)
Out: ['Baby & Kids',
      'Baby & Kids / Toddler & Kids Playroom',
      'Baby & Kids / Toddler & Kids Playroom / Indoor Play',
      'Baby & Kids / Toddler & Kids Playroom / Indoor Play / Rocking
      'Baby & Kids / Toddler & Kids Playroom / Indoor Play / Rocking
```

I'm searching for five tokens now, OR'd (really summed):

```
"Baby & Kids" OR
"Baby & Kids / Toddler & Kids Playroom" OR
"Baby & Kids / Toddler & Kids Playroom / Indoor Play" OR
"Baby & Kids / Toddler & Kids Playroom / Indoor Play / Rocking Horses
"Baby & Kids / Toddler & Kids Playroom / Indoor Play / Rocking Horses
```

Each token of my query has a job. It matches different levels of specificity, and the closer we get the full query classification tree, the higher the score.

Search Term	Job	Score (due to doc freq)	Example of matches
"Baby & Kids"		Lowest	

Enjoy softwaredoug in training course form!

20% off through December 19, 2025 with code ps5

I hope you join me at Cheat at Search with LLMs to learn how to apply LLMs to search applications. Check out this post for a sneak preview.

Building Software Is Learning

THORSTEN BALL · 02 JUN 2026 · SOURCE

An internal note to the Amp team on feedback and shipping faster

A few weeks ago I shared the following as an internal message with the Amp team. I showed it to a friend while talking about feedback loops and he told me to post this publicly. So here we go. Unedited, straight up copy & pasted from our Slack.

You know what's rare?

```
person a: "we need this feature"
person b: "yes, let me build it"
...
person b: "done."
person a: "fantastic, exactly what I wanted."
```

```
  'Furniture / Bedroom Furniture / Dressers & Chests'
  'Outdoor / Outdoor & Patio Furniture / Patio Furniture Sets / Patio
  'Home Improvement / Bathroom Remodel & Bathroom Fixtures / Bathroom
  'Lighting / Wall Lights / Bathroom Vanity Lighting'
  'Kitchen & Tabletop / Kitchen Organization / Food Storage & Caniste
  'School Furniture and Supplies / School Furniture / School Chairs &
  'Baby & Kids / Toddler & Kids Bedroom Furniture / Kids Beds',

If you feel inspired, return many unique values in a list. Be creativ
Cast a wide net with related but diverse categories.

Here's the query to invent categories for:

hobby horse
```

When you actively want hallucinations, you can get by with very small models. You're not relying on the LLM for accuracy, just creativity and close-enough language generation.

Now let's say you do this. It gives you a classification like

```
'Baby & Kids / Toys / Pretend Play & Dress Up / Pretend Play Toys / H
```

More descriptive, closer to the language actually used in the classifications. More likely to give back a real, high quality classification.

That's just a taste. The modern landscape for building large multi-label classifiers has gotten easier than ever thanks to LLMs and embeddings.

Final thoughts

Embeddings are great. But you don't need embeddings to build "semantic search". In fact, it can be counter productive for problems requiring exacting, precise matching. As I've argued, embeddings are the wrong lens to think about the problem.

Maybe you don't need to rebuild everything from scratch, but instead leverage what you're good at. With the addition of LLMs. Don't apologize for living with a more organized, approach to semantic search.

	Matches highest level category		**Baby & Kids** / Toddler & Kids Bedroom Furniture / Kids Headboards
"Baby & Kids / Toddler & Kids Playroom"	Matches next most specific category Getting to "play" stuff for kids, better than just kid stuff. 👍	Low	**Baby & Kids / Toddler & Kids Playroom** / Playroom Furniture / Baby Gyms & Playmats / Folding Baby Gyms & Playmats
…	…	…	…
Baby & Kids / Toddler & Kids Playroom / Indoor Play / Rocking Horses	Matches a very, narrow specific category. Horses!🐴 Even if not "Novelty Rocking Horses" its a defensible "match" to the user.	High	**Baby & Kids / Toddler & Kids Playroom / Indoor Play / Rocking Horses / Traditonal Rocking Horses**

While we keep things in horse → toys → playroom → kids order, we haven't solved the 3rd problem. Limiting matches.

That one's easy. At query time, you can decide how much to limit retrieval. Perhaps you only query the direct node and one parent:

```
"Baby & Kids / ... / Rocking Horses" OR
"Baby & Kids / ... / Rocking Horses / Novelty Rocking Horses"
```

You can tighten / relax in categories that matter to users, ***semantically,*** up a layer to still stay within the realm of "kid play".

If the taxonomy makes sense to users, if it reflects search behaviors (big ifs), they'll understand why you include / exclude certain results.

Importantly, this becomes just ONE component of the ranking / filtering. We can still have other ingredients we can use:

- Keyword matches (for exact mention of properties in titles + description)
- Embedding search (perhaps within the category matches)
- Taxonomies for other entities (colors, materials, etc)

Building a taxonomy: start simple and evolve

Building and maintaining a good taxonomy is no joke. That's why there's a field of taxonomists you can hire. And I don't want to elide the tradeoffs with embeddings here. It's work!

A taxonomy doesn't need to be as deep / complex as what I've laid out. You can start simple. Ask an LLM. Here's one with color.

> Take the 7 primary colors. Then nested under each, create subtypes of each color. Generate a list like a directory trees, in the form `PRIMARY / SECONDARY`

```
RED / Crimson
RED / Scarlet
RED / Burgundy
RED / Maroon
RED / Brick Red
RED / Cherry

ORANGE / Tangerine
ORANGE / Amber
ORANGE / Coral
ORANGE / Burnt Orange
ORANGE / Peach
ORANGE / Apricot
...
```

It's a start.

Generally start simple. Broad, high level, top level categories like `Furniture` or `Baby & Kids`. Then eventually you realize this category has become too large and too diverse. You might see a natural in the content split:

```
Baby & Kids / Toddler & Kids Playroom
Baby & Kids / Toddler & Kids Bedroom Furniture
```

Eventually you get to a point where the distinction between `Novelty Rocking Horse` and `Classic Rocking Horse` matters for some reason.

You may see a flaw: an assumption each product sits within a single category. That categories are mutually exclusive. Reality, of course, is messy: some products may sit between categories. You can imagine a small goofy looking couch that could reasonably tagged under `Baby & Kids / Toddler & Kids Bedroom Furniture` but also `Living Room / Loveseats`. In Fashion, some activewear might also be underwear. And what about athleisure?

One solution could be to double categorize an item. Give it both classifications - with some weight preferring one categorization over another.

A final concern - there's a big difference between the official pure way of classifying furniture and how a search user might categorize the products. The exacting, furniture-expert taxonomy might have very sophisticated (and accurate) ways to classify furniture. A messier classification that matches how users think about things might be more useful in the end.

Classifying into taxonomies has never been easier

Building taxonomy classifiers with LLMs - something I've built an entire class for, and could be a post on its own. But lets do a quick walkthrough of some simpler approaches.

Embeddings actually help us here. The sweet spot of embeddings in search might be building better classifiers, not in direct ranking and retrieval.

Perhaps our products fit in a few thousand unique categories. Imagine we encode an embedding each of these in a small in-memory array of vectors. We'll use it to lookup categories for our queries.

Now take a query "hobby horse". Encode the query into an embedding, and find the most similar category embedding. Maybe this retrieves `Baby & Kids / Toddler & Kids Playroom / Indoor Play / Rocking Horses`.

Not accurate enough? Let's take it up a notch.

We could ask an LLM to hallucinate a fake label before categorizing. Ask an LLM the following:

```
Be creative and hallucinate a set of classifications for the query be
look like the real classifications.

Product classifications might look like:

  'Furniture / Living Room Furniture / Coffee Tables & End Tables / C
  'Décor & Pillows / Decorative Pillows & Blankets / Throw Pillows'
```